Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

O que o enunciado pede e o que você entrega. Não explique a implementação aqui.

Nesta atividade, o enunciado pedia o algoritmo do banqueiro (Silberschatz, 10ª ed., P-45) em C, sem fixar o número de clientes nem de recursos: os recursos chegam pelos argumentos e os clientes vêm das linhas do customer.txt. O programa lê o customer.txt e o commands.txt, executa os comandos e grava o resultado no result.txt, encerrando com uma mensagem clara sempre que a entrada é inválida. Estou entregando o banker.c, um Makefile (make e make clean), um script com 18 cenários de teste, o evidencias.log e o README. Com a entrada do enunciado, o result.txt que o programa gera ficou idêntico ao esperado.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
Implementado em C, roda em Linux/Unix/macOS	OK	Compilei com gcc no Ubuntu (WSL); está no evidencias.log
make gera o executável banker	OK	make clean && make no log
make clean remove os arquivos compilados	OK	make clean apaga o banker e o result.txt
Número de recursos definido pelos argumentos	OK	./banker 10 5 7 (3 recursos) e ./banker 6 6 6 6 (4 recursos)
Número de clientes = número de linhas do customer.txt	OK	Testei com 5 clientes (enunciado) e com 3 clientes
Lê os comandos RQ, RL e * do commands.txt	OK	Teste "Exemplo do enunciado"
Gera result.txt igual ao exemplo	OK	diff result.txt esperado.txt não mostra diferenças
RQ negado por exceder o máximo	OK	Linha "exceed its maximum allocation" no exemplo
RQ negado por falta de recursos	OK	Linha "are not enough" no exemplo
RQ negado por estado inseguro	OK	Linha "result in an unsafe state" no exemplo
Erro: nº de argumentos diferente do customer.txt	OK	./banker 10 5 e ./banker 10 5 7 3
Erro: nº de recursos diferente no commands.txt	OK	Teste com RQ 0 0 2
Erro: customer.txt ou commands.txt inexistente	OK	Testes "nao existe" no log
Erro: formato inválido nos arquivos	OK	Letra, espaço no lugar de vírgula, comando desconhecido, cliente inexistente e valor negativo
Não falha (sem crash) com entrada inválida	OK	Todos os casos de erro saem com código 1 e mensagem

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Todos os comandos rodam dentro da pasta alvs/, em Linux ou macOS (no Windows, pelo WSL).

Compilar:

```
cd alvs
make clean
make
```

Executar (exemplo):

Antes, coloque na pasta alvs/ o customer.txt e o commands.txt que o professor forneceu.

```
./banker 10 5 7

cat result.txt
```

Comparar com a saída esperada (salve o result.txt do professor como esperado.txt; se estiver tudo certo, o comando não imprime nada):

```
diff result.txt esperado.txt
```

Rodar os 18 cenários de teste (o script cria as próprias entradas numa pasta temporária, então não depende dos arquivos do professor):

```
bash testes.sh
```

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades:

1. **banker.c → lê os argumentos e os dois arquivos, valida tudo, executa RQ, RL e * e grava o result.txt**
2. **Makefile → make compila com -Wall -Wextra -std=c99; make clean remove o banker e o result.txt**
3. **testes.sh → roda 18 cenários (2 corretos e 16 de erro), cada um com as suas próprias entradas**
4. **evidencias.log → gravação da sessão de terminal feita com o script**

Decisões técnicas:

1. **Vetores e matrizes com malloc/realloc → o número de clientes e de recursos só aparece em tempo de execução → não há limite fixo, e a memória é liberada no fim do main**
2. **Validar o commands.txt inteiro antes de executar (duas passadas com rewind) → o enunciado pede que todos os parâmetros sejam checados antes → um erro na última linha impede a execução e o result.txt não fica pela metade**
3. **No RQ, checar primeiro o máximo, depois o disponível e por último a segurança → é a ordem do algoritmo do livro e a que reproduz a saída do exemplo → o teste de segurança, que é o mais caro, só roda quando o pedido é possível**
4. **Alocar provisoriamente e desfazer se o estado ficar inseguro → o teste de segurança precisa do estado como ele ficaria depois da alocação → as mesmas funções servem para o estado real e para a simulação**
5. **Negar RL maior que o alocado, com mensagem própria → o enunciado não define esse caso, e aceitar deixaria a alocação negativa → nova mensagem: "The customer X release ... was denied because exceed its allocation"**
6. **Mensagens de erro no stderr e saída com código 1 → separa erro de resultado e deixa scripts detectarem a falha → o testes.sh mostra o código de saída de cada caso**
7. **Remover o \r do fim das linhas → arquivos criados no Windows terminam as linhas com CRLF → entradas vindas do Windows não são rejeitadas por engano**

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Usei uma única estratégia do começo ao fim. Os problemas que apareceram foram de ambiente (Windows × Linux), não do algoritmo em si

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Validação completa + banqueiro do livro	Ler e validar toda a entrada primeiro e só depois executar os comandos, usando o algoritmo de segurança do livro e estruturas dinâmicas	Não houve troca
S2			
S3			

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

Observação: cada linha aponta para um commit no GitHub, para a sessão gravada no evidencias.log (11/11, 15:38−15:41) ou para um print da seção 6.2.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Copiar os arquivos para o repositório clonado com cp -r e rm -f	Falhou: Copy-Item e Remove-Item recusaram os parâmetros	Comandos de Bash executados no PowerShell, onde cp e rm são outros comandos com outros parâmetros	09/11	—
2	S1	Criar o Makefile (make e make clean) e o .gitignore	OK	—	09/11 15:42	commit 70dc403
3	S1	Implementar o banker.c: leitura e validação dos argumentos e dos dois arquivos, comandos RQ, RL e * e algoritmo de segurança	OK	—	10/11 14:11	commit 1a323fc
4	S1	Commitar o banker.c no Windows	Parcial: commit feito, com aviso "LF will be replaced by CRLF"	core.autocrlf=true converteria os arquivos para CRLF, o que quebra o testes.sh no Linux; resolvido com .gitattributes (eol=lf)	11/11 14:49	commit 7dac14f
5	S1	Escrever o README com compilação,	OK	—	11/11 14:49	commit a8b467f

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
		execução e testes				
6	S1	Compilar do zero (make clean && make)	OK, sem nenhum warning	—	11/11, sessão 15:38–15:41	evidencias.log
7	S1	Rodar ./banker 10 5 7 e comparar com a saída do professor (diff)	OK, IDENTICO	—	11/11, sessão 15:38–15:41	evidencias.log, Figura 2
8	S1	Rodar ./banker 10 5 (argumentos a menos que no customer.txt)	OK: "foram passados 2 argumentos (recursos), mas a linha 1 do customer.txt tem 3 recursos"	—	11/11, sessão 15:38–15:41	evidencias.log, Figura 3
9	S1	Renomear o commands.txt e rodar ./banker 10 5 7	OK: "nao foi possivel abrir o arquivo commands.txt"	—	11/11, sessão 15:38–15:41	evidencias.log, figura 1
10	S1	bash testes.sh (18 cenários: 2 corretos e 16 inválidos)	OK: 16 erros com código de saída 1 e sem gerar result.txt	—	11/11, sessão 15:38–15:41	evidencias.log
11	S1	Commitar o testes.sh e o evidencias.log	OK	—	11/11 15:41	commit fcb1ccc

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

A minha sessão de validação foi gravada em alvs/evidencias.log (que vai dentro do .tar e no GitHub). Rodei estes comandos no Ubuntu (WSL), dentro de alvs/:

```
script -a evidencias.log
date; whoami; pwd
make clean
make
./banker 10 5 7
cat result.txt
diff result.txt esperado.txt && echo IDENTICO
./banker 10 5
bash testes.sh
exit
```

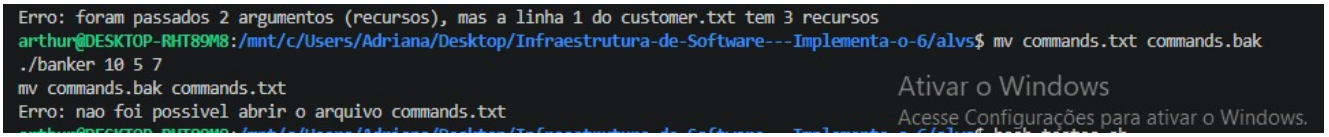
6.2 Prints

- Erro principal: o momento do bug mais relevante da semana.
- Validação final: teste passando / saída correta.

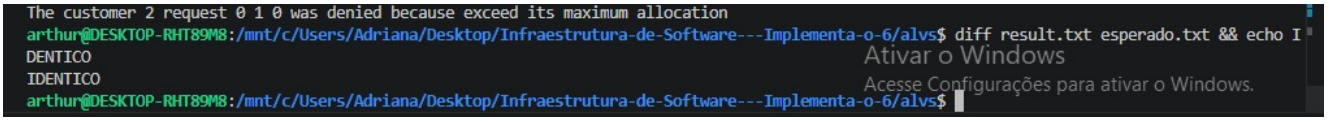
Prints extras só se algo excepcional aconteceu e o log sozinho não explica, você precisa ter esse feeling, mas posso dar uma ajuda (ex.: comportamento visual, crash com stack trace longo).

Erro principal: o erro mais relevante foi o da tentativa 1 (cp -r e rm -f no PowerShell). Ele não tem print porque o terminal foi fechado antes. As figuras abaixo mostram os casos de erro tratados pelo programa.

Caso limite (tentativa 9 do diário): commands.txt ausente, o programa encerra com "nao foi possivel abrir o arquivo commands.txt" (figura 1)



Validação final: diff sem diferenças e IDENTICO na tela, seguido do cat result.txt.(figura 2)



Caso inválido: ./banker 10 5 mostrando a mensagem sobre o número de argumentos (figura 3).



7. Uso de IA

Obrigatório preencher mesmo se a resposta for "não utilizei".

Onde usei IA (2–3 linhas):
Usei o Claude para me ajudar e guiar para fazer o banker.c, Makefile, testes.sh e o README. Também usei a IA para entender o algoritmo, configurar o Git.

Prompts principais (liste, sem história):
– me ajude e me guie para eu Implementar a implementaçao 5 da forma mais simples possível, em português, organizado em etapas de commit

– Explicar a implementação de forma geral

– Passo a passo para criar o repositório no GitHub, clonar e fazer os commits

– Revisar o que entregar

O que validei manualmente e como:
Compilei do zero no WSL; comparei o result.txt com o do professor usando diff; rodei casos de erro à mão; segui o exemplo do enunciado no papel, comando por comando, conferindo available, allocation e need

8. Reflexão Final (5–8 linhas)

O que você melhoraria com mais tempo, principais decisões técnicas e erros que evitaria na próxima. Tudo junto, sem subseções separadas.

Reflexão:

Para mim, a decisão mais importante foi validar toda a entrada antes de executar qualquer comando: assim, um erro na última linha do commands.txt nunca deixa um result.txt pela metade. A segunda foi alocar de forma provisória e desfazer quando o estado fica inseguro, o que deixou o RQ curto e fiel ao livro. O que mais me custou tempo não foi o algoritmo, e sim o ambiente: o final de linha CRLF do Windows e comandos do Git Bash rodados no PowerShell. Da próxima vez, vou compilar e testar no Linux desde o primeiro minuto e já começar a sessão dentro do script, para o log cobrir o desenvolvimento inteiro. Com mais tempo, trocaria a dupla leitura do commands.txt por uma lista de comandos em memória e faria o testes.sh comparar as saídas sozinho.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	ok
Rodei os testes e evidencias.log foi gerado	ok
Tenho prints obrigatórios	ok
Testei pelo menos um caso limite e um caso inválido	ok
Preenchi a seção de uso de IA	ok
Revisei o relatório e removi frases genéricas / vazias	ok

10. Se eu tivesse mais 2 horas

Aplique as instruções abaixo de forma objetiva e verificável, evitando texto genérico.

Escreva 5-10 linhas sobre o que você melhoraria (robustez, testes, warnings, performance, organização).

1. Testes automáticos: fazer o testes.sh comparar cada result.txt com um arquivo esperado e imprimir PASSOU ou FALHOU, em vez de depender de alguém lendo a saída.

2. Memória: rodar todos os cenários no valgrind --leak-check=full. Hoje, nos caminhos de erro, o exit(1) sai sem liberar as matrizes; o sistema recupera a memória, mas o valgrind acusaria.

3. Robustez: checar o retorno de malloc e realloc e tratar linhas com mais de 1024 caracteres, que hoje seriam lidas em duas partes.

4. Validação extra: recusar um cliente cujo máximo seja maior que o total de instâncias do recurso, já que esse cliente nunca conseguiria terminar.

5. Organização: separar leitura, algoritmo e saída em entrada.c, banqueiro.c e saida.c, com um .h em comum.

6. Desempenho: guardar os comandos já validados numa lista em memória, em vez de ler o commands.txt duas vezes.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.